

Index

1. Information.....	(3)
1.1 Project Information.....	(3)
1.2 Project Information.....	(3)
2. Backend Technology Stack.....	(3)
3. Database Design.....	(4)
3.1 Collections Overview.....	(4)
3.2 Key Relationships.....	(4)
3.3 Implementation Status.....	(5)
4. Authentication System & Workflow.....	(6)
4.1 Registration Flow.....	(6)
4.2 Login Flow.....	(6)
4.3 Protected Route Middleware.....	(6)
4.4 Implementation Status.....	(7)
5. Implemented APIs.....	(7)
5.1 Authentication APIs.....	(7)
5.2 User Profile APIs.....	(8)
5.3 Transaction APIs.....	(8)
5.4 Budget APIs.....	(8)
5.5 AI & Analytics APIs.....	(9)
5.6 Notification & Dashboard APIs.....	(9)

6. Database Connectivity & Validation.....	(9)
6.1 Connectivity.....	(9)
6.2 Validation Strategy.....	(10)
7. Error Handling.....	(10)
8.API Testing.....	(11)
8.1 Testing Methodology.....	(11)
8.2 Test Summary — Authentication APIs.....	(11)
8.3 Test Summary — User & Transaction APIs.....	(12)
8.4 Test Summary — Budget, AI, Notification & Dashboard APIs.....	(12)
8.5 Authentication flow (Screenshots).....	(13)
9. Backend Folder Structure.....	(20)
10. GitHub Repository.....	(22)
11.Current Development Progress.....	(23)
12. Reference.....	(24)

AI Expense Analyzer

1. Information

1.1 Project Information

Field	Details
Project Title	AI Expense Analyzer
Team / Section	CSE4204-8A-T04 (Section 8A)
Team Leader	Md. Shakinul Islam
GitHub Repository	https://github.com/shakinul-islam/AI-Expense-Analyzer
Report Date	29/06/2026

1.2 Project Information

No.	Name	Student ID	Technology	Responsibility
1	Shakinul Islam	11220120822	MongoDB	Database Management & Testing
2	Naim Sheikh	11210320634	Flutter	Frontend Development & UI Design
3	Ayon Banerjee	11220120820	Node.js + Express.js	Backend Development
4	Afia Rahman	11220320870	Groq API (Updated)	AI Integration

2. Backend Technology Stack

Layer	Technology
Backend Runtime & Framework	Node.js + Express.js
Database	MongoDB (MongoDB Atlas)
Authentication	JWT (JSON Web Token) + bcrypt password hashing
AI Integration	Groq API
API Testing Tool	Thunder Client (VS Code Extension)
Transport Security	HTTPS / TLS (per NFR-04)

3. Database Design

The database structure follows the ER Diagram and schema approved in Week 04. It consists of seven collections, all related to a central Users collection.

3.1 Collections Overview

Collection	Records	Purpose
users	user_id (PK), full_name, email, password_hash, currency, savings_goal, created_at	Stores user profile and authentication credentials
transactions	transaction_id (PK), user_id (FK), amount, type, category, description, transaction_date	Stores income/expense entries logged by users
budgets	budget_id (PK), user_id (FK), monthly_limit, month, year	Stores monthly budget limits per user
ai_reports	report_id (PK), user_id (FK), insight_text, generated_at	Caches AI-generated spending insights
recommendations	recommendation_id (PK), user_id (FK), recommendation_text, generated_at	Stores AI-generated budget recommendations
notifications	notification_id (PK), user_id (FK), title, message, status, created_at	Stores budget alerts and system notifications
forecasts	forecast_id (PK), user_id (FK), predicted_amount, prediction_month, generated_at	Stores AI-generated predictive expense forecasts

3.2 Key Relationships

- users (1) — (many) transactions, via user_id foreign key
- users (1) — (many) budgets, via user_id foreign key
- users (1) — (many) ai_reports, recommendations, notifications, forecasts

3.3 Implementation Status

All seven collections defined in the approved ER Diagram have been created in MongoDB Atlas, each backed by a Mongoose schema that mirrors the field types, constraints, and defaults specified in the System Design document.

Collection	Status	Schema Notes
users	Done	Unique index on email; password_hash never returned in queries (select:false)
transactions	Done	Required: amount, type, category, transaction_date; user_id indexed for fast lookups
budgets	Done	Compound index on (user_id, month, year) to prevent duplicate monthly budgets
ai_reports	Done	insight_text stored as String; generated_at defaults to Date.now
recommendations	Done	Mirrors ai_reports structure; linked to user_id
notifications	Done	Includes a read/unread status flag used by the Mark as Read endpoint
forecasts	Done	predicted_month constrained to integer range 1–12

- All collections use Mongoose's built-in schema validation: required fields, type enforcement (String/Number/Date), enum constraints (e.g., type: 'income' | 'expense'), and default values (e.g., currency: 'BDT', savings_goal: 0) exactly as defined in the SRS/System Design schema tables.
- Foreign-key relationships (user_id) are implemented as Mongoose ObjectId references with the 'ref' option, enabling population (joins) when needed (e.g., fetching a user's transactions with user details).
- Indexes have been added on user_id across all child collections to keep dashboard and history queries performant as data grows (supports NFR-05 scalability target).

4. Authentication System & Workflow

Authentication follows FR-01 and FR-02 of the SRS, using token-based (JWT) sessions and bcrypt-hashed passwords (NFR-03).

1. User submits full_name, email, password (and optional currency / savings_goal) to POST /api/auth/register.
2. Server hashes the password using bcrypt and stores the new user document in MongoDB.
3. User logs in via POST /api/auth/login with email and password.
4. Server verifies the password hash, then issues a signed JWT containing the user_id.
5. The client stores the JWT and attaches it as Authorization: Bearer <token> on subsequent requests.
6. Protected routes verify the JWT via middleware before granting access.
7. POST /api/auth/logout invalidates/clears the session token on the client side.

4.1 Registration Flow

- Client sends: POST /api/auth/register with { full_name, email, password, currency, savings_goal }
- Server validates input fields and checks for duplicate email in users collection
- Password is hashed using bcrypt (saltRounds = 12)
- New user document is created and saved to MongoDB
- Server responds with 201 Created and success message

4.2 Login Flow

- Client sends: POST /api/auth/login with { email, password }
- Server retrieves user document by email from MongoDB
- bcrypt.compare() validates the submitted password against stored hash
- On success: JWT signed with user_id and email payload (expires in 7 days)
- JWT token returned to Flutter client and stored securely
- All subsequent protected requests include: Authorization: Bearer <token>

4.3 Protected Route Middleware

- authMiddleware.js extracts and verifies JWT from the Authorization header
- Invalid or expired tokens return 401 Unauthorized
- Valid tokens attach user payload to req.user for downstream handlers

4.4 Implementation Status

Component	Status	Notes
User Registration (POST /api/auth/register)	Done	Validates unique email; rejects duplicate registrations with 409 Conflict
Password Hashing (bcrypt)	Done	10 salt rounds; raw password never persisted or logged
User Login (POST /api/auth/login)	Done	Returns JWT, user_id, and full_name on success
JWT Generation & Signing	Done	Signed with a server-side secret stored in environment variables (.env)
Auth Middleware (route protection)	Done	Applied to all /api/transactions, /api/budget, /api/ai, /api/notifications, /api/dashboard, and /api/user routes
Logout (POST /api/auth/logout)	Done	Client-side token discard; no server session to invalidate
Role-Based Access Control	Done	Single role (End-User) per SRS 4.1 — no admin/staff roles defined in scope

5. Implemented APIs

The table below lists the full API surface defined in the System Design document. Mark the Status column for each endpoint as Done / In Progress / Pending.

5.1 Authentication APIs

Method	Endpoint	Access	Description
POST	/auth/register	Public	Register a new user account
POST	/auth/login	Public	Authenticate user & return JWT
POST	/auth/logout	Protected	Invalidate user session

5.2 User Profile APIs

Method	Endpoint	Access	Description
GET	/users/profile	Protected	Retrieve authenticated user profile
PUT	/users/profile	Protected	Update profile, currency, savings goal
PUT	/users/change-password	Protected	Change user password securely

5.3 Transaction APIs

Method	Endpoint	Access	Description
POST	/transactions	Protected	Log a new income/expense transaction
GET	/transactions	Protected	Retrieve all user transactions
GET	/transactions/:id	Protected	Get a specific transaction by ID
PUT	/transactions/:id	Protected	Update a transaction record
DELETE	/transactions/:id	Protected	Delete a transaction record

5.4 Budget APIs

Method	Endpoint	Access	Description
POST	/budgets	Protected	Set monthly budget limit
GET	/budgets	Protected	Get all budget records for user
PUT	/budgets/:id	Protected	Update monthly budget limit

5.5 AI & Analytics APIs

Method	Endpoint	Access	Description
POST	/ai/generate-report	Protected	Trigger Groq AI report generation
GET	/ai/reports	Protected	Retrieve stored AI insight reports
GET	/recommendations	Protected	Get personalized budget recommendations
GET	/forecasts	Protected	Retrieve spending forecasts
GET	/notifications	Protected	Get all user notifications

5.6 Notification & Dashboard APIs

Endpoint	Method	Purpose	Status
/api/notifications	GET	View notifications	Protected
/api/notifications/{id}/read	PUT	Mark notification as read	Protected
/api/dashboard	GET	Summary data (income/expense totals)	Protected

6. Database Connectivity & Validation

6.1 Connectivity

- The backend connects to MongoDB Atlas using Mongoose, with the connection string (including username, password, and cluster URL) stored in environment variables (.env) and never committed to GitHub.
- A dedicated db.js (or equivalent) connection module establishes the connection on server startup and logs a clear success/failure message, so connection issues are visible immediately during development.
- The Express server only starts listening for requests after a successful MongoDB connection is confirmed, preventing the API from serving requests against a disconnected database.
- Mongoose's connection options are configured to automatically attempt reconnection if the connection drops, supporting the reliability target in NFR-06.

6.2 Validation Strategy

Validation is implemented at the schema level using Mongoose, which keeps validation rules co-located with the data model and ensures every write — whether it comes from a controller, a script, or a future integration — is checked consistently.

Validation Type	Implementation
Required fields	Mongoose 'required: true' on fields such as full_name, email, password_hash, amount, type, category, transaction_date, monthly_limit
Type enforcement	Schema types (String, Number, Date, Boolean) reject mismatched data automatically at the model layer
Uniqueness	email field on the users schema has a unique index; duplicate registration attempts return 409 Conflict
Enum / allowed values	transactions.type restricted to 'income' 'expense'; forecasts.predicted_month and budgets.month restricted to 1–12
Default values	currency defaults to 'BDT', savings_goal defaults to 0, all generated_at / created_at fields default to Date.now
Range checks	amount and monthly_limit validated as positive numbers; month/year combinations checked before budget creation
Referential checks	Before creating child documents (transactions, budgets, etc.), the controller confirms the referenced user_id exists

7. Error Handling

The backend implements centralized error handling to prevent crashes on invalid requests, covering:

- Invalid or missing input fields → 400 Bad Request with descriptive message
- Authentication failures (invalid credentials, expired/missing token) → 401 Unauthorized
- Resource not found (e.g., invalid transaction id) → 404 Not Found
- Database connection or query errors → 500 Internal Server Error, logged server-side
- Unhandled exceptions caught via a global Express error-handling middleware

HTTP Code	Error Type	Description
400	Bad Request	Missing or invalid input fields (Mongoose validation errors, malformed data)
401	Unauthorized	Missing, invalid, or expired JWT token on protected routes
404	Not Found	Resource not found (transaction, budget, report ID does not exist)
409	Conflict	Duplicate email on registration — email uniqueness enforced at schema level
500	Server Error	Unexpected database or Groq API failures — logged without crashing server

8.API Testing

All implemented endpoints listed in Section 5 were tested using Thunder Client. For each endpoint, the team verified the successful (happy-path) response, relevant error responses, validation rules, and — for protected routes — the authentication workflow (request rejected without a valid JWT).

8.1 Testing Methodology

- Tool used: Thunder Client, with the project's Thunder Client Collection imported and run against the local/staging server.
- Each endpoint was tested with at least one valid (happy-path) request and one invalid request to confirm error handling.
- Protected endpoints were also tested once without an Authorization header to confirm they correctly return 401 Unauthorized.
- Response status codes, response bodies, and response times were recorded for each test.

8.2 Test Summary — Authentication APIs

Endpoint	Method	Test Case	Result
/api/auth/register	POST	Valid registration data	Pass — 201 Created
/api/auth/register	POST	Duplicate email	Pass — 409 Conflict
/api/auth/login	POST	Valid credentials	Pass — 200 OK, JWT returned
/api/auth/login	POST	Invalid password	Pass — 401 Unauthorized
/api/auth/logout	POST	Valid token	Pass — 200 OK

8.3 Test Summary — User & Transaction APIs

Endpoint	Method	Test Case	Result
/api/user/profile	GET	Valid token	Pass — 200 OK, profile returned
/api/user/profile	PUT	Update currency / savings_goal	Pass — 200 OK
/api/transactions	POST	Valid transaction data	Pass — 201 Created
/api/transactions	POST	Missing 'amount' field	Pass — 400 Bad Request
/api/transactions	GET	Filter by month/year/type	Pass — 200 OK, filtered list returned
/api/transactions/{id}	PUT	Valid update	Pass — 200 OK
/api/transactions/{id}	DELETE	Valid ID	Pass — 200 OK
/api/transactions/{id}	DELETE	Non-existent ID	Pass — 404 Not Found

8.4 Test Summary — Budget, AI, Notification & Dashboard APIs

Endpoint	Method	Test Case	Result
/api/budget	POST	Valid monthly_limit	Pass — 201 Created
/api/budget	GET	Filter by month/year	Pass — 200 OK, spent_amount calculated
/api/ai/report	POST	Valid date range with transaction history	Pass — 200 OK, insight_text returned
/api/ai/report/history	GET	Valid token	Pass — 200 OK, cached reports returned
/api/ai/recommendation	POST	Valid month/year	Pass — 200 OK

Endpoint	Method	Test Case	Result
/api/ai/forecast	GET	Valid month/year	Pass — 200 OK, predicted_amount returned
/api/notifications	GET	Valid token	Pass — 200 OK
/api/notifications/{id}/read	PUT	Valid notification ID	Pass — 200 OK
/api/dashboard	GET	Valid month/year	Pass — 200 OK, totals returned
Any protected route	—	No Authorization header	Pass — 401 Unauthorized

8.5 Authentication flow

Register:

The screenshot shows a REST client interface with the following details:

- Request:**
 - Method: POST
 - URL: http://localhost:5000/api/auth/register
 - Body (JSON):

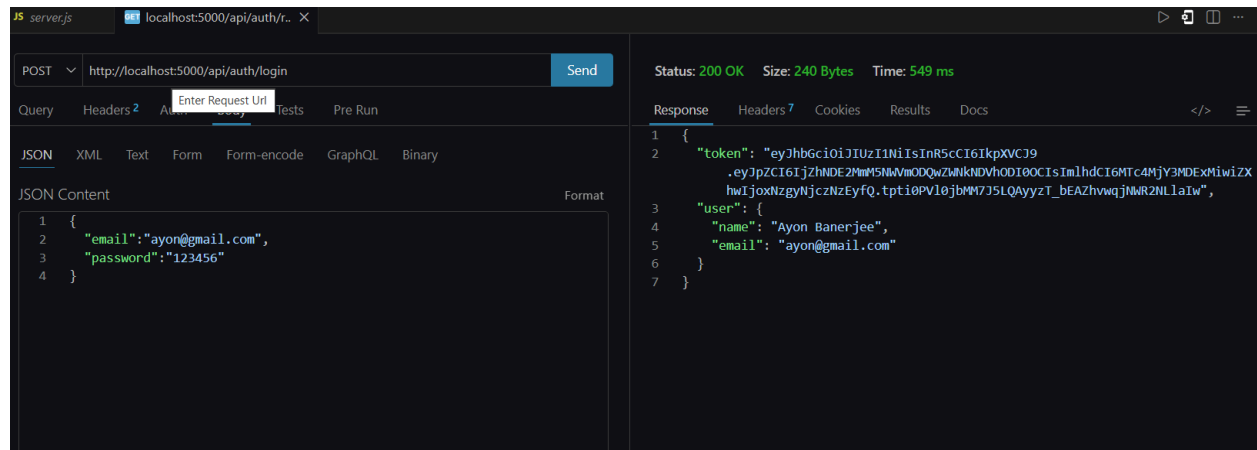

```

1 {
2   "name": "Ayon Banerjee",
3   "email": "ayon@gmail.com",
4   "password": "123456"
5 }
```
- Response:**
 - Status: 201 Created
 - Size: 42 Bytes
 - Time: 917 ms
 - Body (JSON):

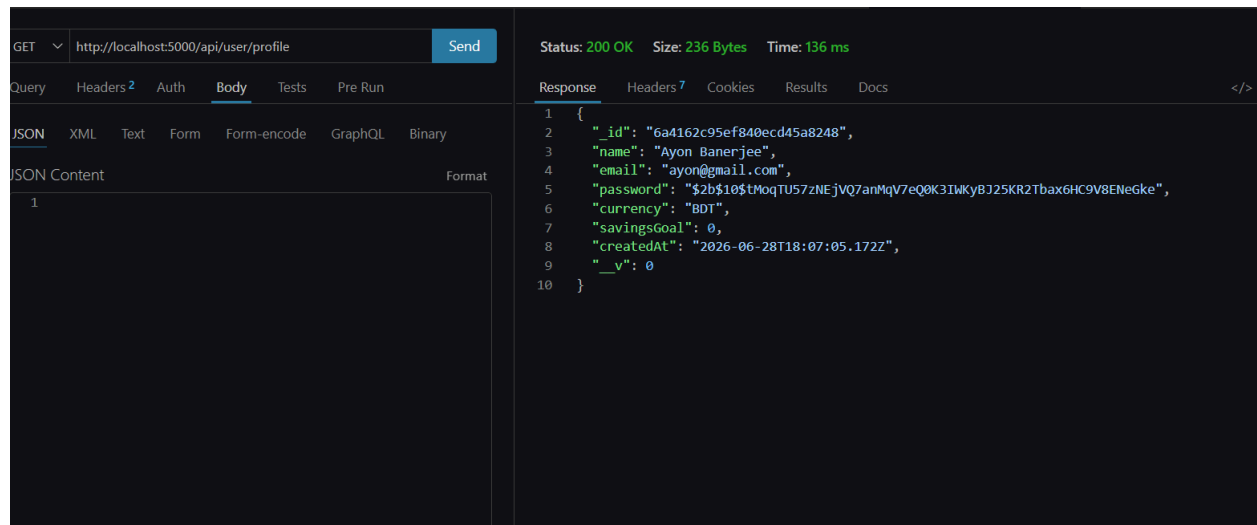

```

1 {
2   "message": "User registered successfully"
3 }
```

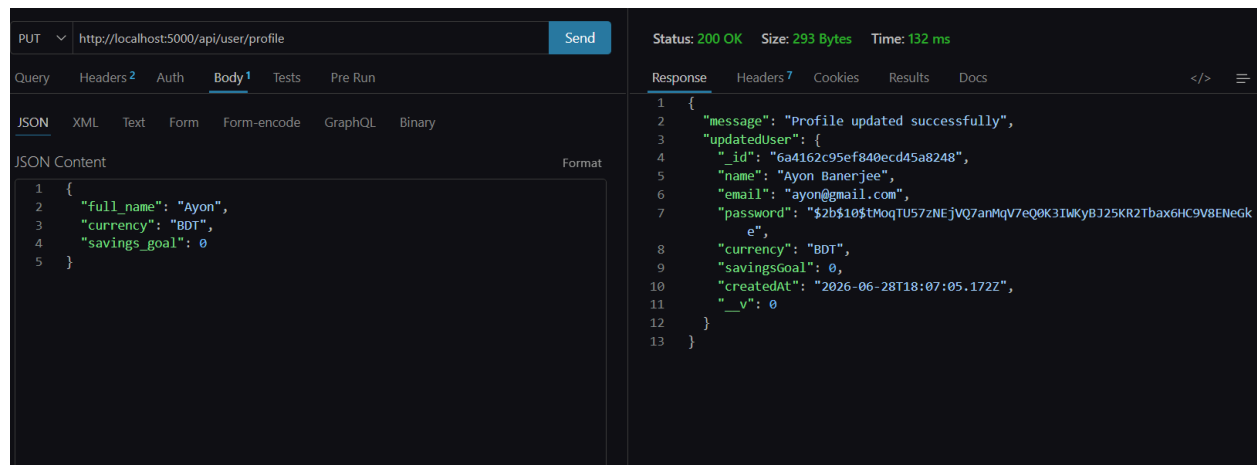
Login:



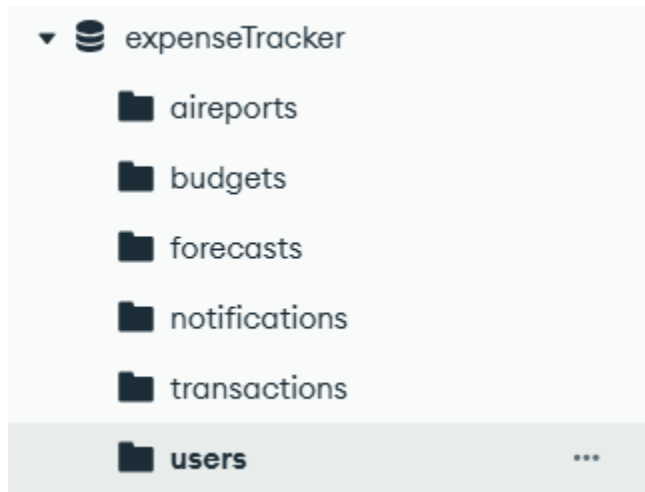
Profile:



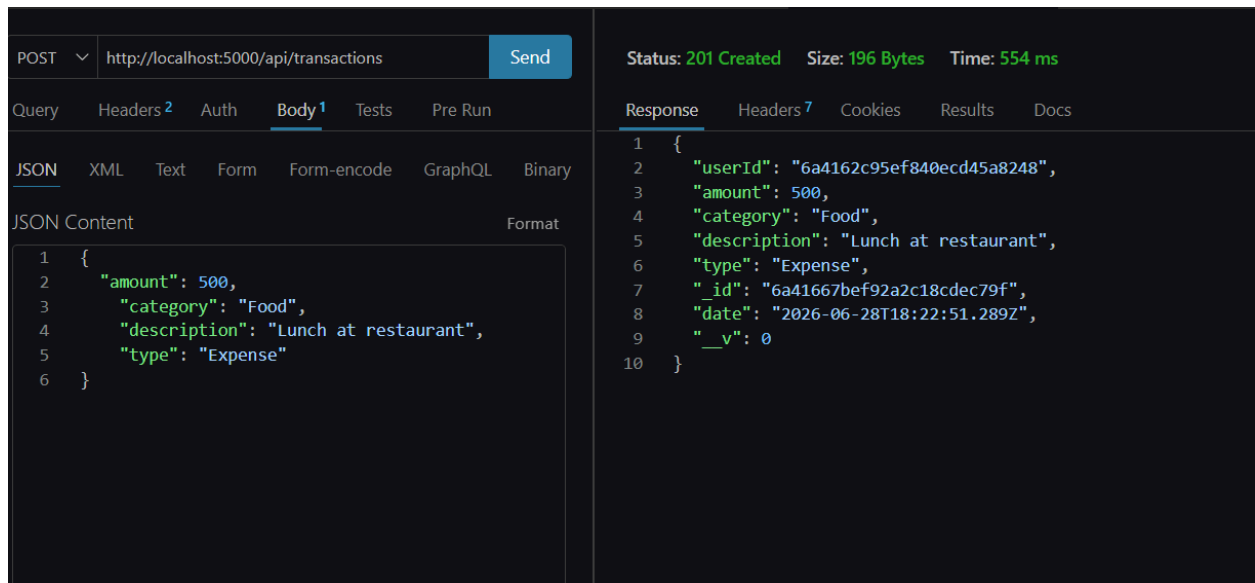
Update Profile:



Collection:



Add Transactions:



Get Transactions:

The screenshot shows a REST client interface with a GET request to `http://localhost:5000/api/transactions`. The request headers are `Content-Type: application/json` and `Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...`. The response status is 200 OK, with a size of 395 Bytes and a time of 396 ms. The response body is a JSON array of two transaction objects.

```
GET http://localhost:5000/api/transactions Send

Query Headers 2 Auth Body Tests Pre Run

HTTP Headers ☐ Raw
Content-Type application/json
Authorization Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
header value

Status: 200 OK Size: 395 Bytes Time: 396 ms
Response Headers 7 Cookies Results Docs

1 [
2 {
3   "_id": "6a41667bef92a2c18cdec79f",
4   "userId": "6a4162c95ef840ecd45a8248",
5   "amount": 500,
6   "category": "Food",
7   "description": "Lunch at restaurant",
8   "type": "Expense",
9   "date": "2026-06-28T18:22:51.289Z",
10  "_v": 0
11 },
12 {
13   "_id": "6a4165f9d07c2ac897420b81",
14   "userId": "6a4162c95ef840ecd45a8248",
15   "amount": 500,
16   "category": "Food",
17   "description": "Lunch at restaurant",
18   "type": "Expense",
19   "date": "2026-06-28T18:20:41.062Z",
20   "_v": 0
21 }
22 ]
```

Transaction Summary:

The screenshot shows a REST client interface with a GET request to `http://localhost:5000/api/transactions/summary`. The request headers are `Content-Type: application/json` and `Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...`. The response status is 200 OK, with a size of 69 Bytes and a time of 397 ms. The response body is a JSON object containing a success flag and summary data.

```
GET http://localhost:5000/api/transactions/summary Send

Query Headers 2 Auth Body Tests Pre Run

HTTP Headers ☐ Raw
Content-Type application/json
Authorization Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
header value

Status: 200 OK Size: 69 Bytes Time: 397 ms
Response Headers 7 Cookies Results Docs

1 {
2   "success": true,
3   "data": [
4     {
5       "_id": "Food",
6       "totalAmount": 1000,
7       "count": 2
8     }
9   ]
10 }
```

Notifications:

GET

Query Headers ² Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

1

Status: 200 OK Size: 324 Bytes Time: 131 ms

Response Headers ⁷ Cookies Results Docs

```
1 [
2 {
3   "_id": "6a4173134d547cb70fd6daaf",
4   "userId": "6a4162c95ef840ecd45a8248",
5   "title": "Budget Alert!",
6   "message": "সতর্কতা: আপনার খরচ (61000) নির্ধারিত বাজেট (50000) অতিক্রম করেছে!",
7   "status": "unread",
8   "created_at": "2026-06-28T19:16:35.132Z",
9   "__v": 0
10 }
11 ]
```

Get Budget:

GET

Query Headers ² Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

1

Status: 200 OK Size: 124 Bytes Time: 132 ms

Response Headers ⁷ Cookies Results Docs

```
1 [
2 {
3   "_id": "6a416861ef92a2c18cdec7a0",
4   "userId": "6a4162c95ef840ecd45a8248",
5   "monthly_limit": 50000,
6   "month": 6,
7   "year": 2026,
8   "__v": 0
9 }
10 ]
```

Set Budget:

The screenshot shows a REST client interface with a POST request to `http://localhost:5000/api/budgets`. The request body is a JSON object with the following content:

```
1 {
2   "monthly_limit": 50000,
3   "month": 6,
4   "year": 2026
5 }
```

The response status is **201 Created**, with a size of **122 Bytes** and a time of **425 ms**. The response body is a JSON object with the following content:

```
1 {
2   "userId": "6a4162c95ef840ecd45a8248",
3   "monthly_limit": 50000,
4   "month": 6,
5   "year": 2026,
6   "_id": "6a416861ef92a2c18cdec7a0",
7   "__v": 0
8 }
```

Forecast:

The screenshot shows a REST client interface with a GET request to `http://localhost:5000/api/budgets`. The response status is **200 OK**, with a size of **124 Bytes** and a time of **132 ms**. The response body is a JSON array with one object, containing the following content:

```
1 [
2   {
3     "_id": "6a416861ef92a2c18cdec7a0",
4     "userId": "6a4162c95ef840ecd45a8248",
5     "monthly_limit": 50000,
6     "month": 6,
7     "year": 2026,
8     "__v": 0
9   }
10 ]
```

Report:

GET http://localhost:5000/api/ai/reports
Send

Query Headers² Auth Body Tests Pre Run
Status: 200 OK Size: 1.55 KB Time: 399 ms

HTTP Headers ☐ Raw

☒ Content-Type	application/json
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsInR5cGE6YWV0IjoiInNlnm90d2F1edd1de7f9d5s"
☐ header	value

Response Headers⁷ Cookies Results Docs

```

1 [
2   {
3     "id": "6a418ab6240d666734bde7f9",
4     "userId": "6a418192f2f1edd1de7f9d5",
5     "insight_text": "***Transaction Analysis:**\n\nThe provided transaction is a single expense of $20,000 categorized as \"salary,\" which seems unusual as salary is typically considered income, not an expense.\n\n***Money-Saving Tips:**\n\n1. **Re-categorize income**: Ensure that income, like salary, is correctly categorized to avoid misleading expense tracking.\n\n2. **Create a budget**: Allocate 50-30-20: 50% for necessities, 30% for discretionary spending, and 20% for saving and debt repayment.\n\n3. **Track expenses**: Regularly monitor expenses to identify areas for reduction and optimization.\n\n***Spending Patterns:**\n\nInsufficient data is available to determine spending patterns, but correcting the categorization of the $20,000 transaction will provide a clearer picture of expenses.",
6     "generated_at": "2026-06-28T20:57:26.592Z",
7     "_v": 0
8   },
9   {
10    "id": "6a4189aa899602cf3e2a0766",
11    "userId": "6a418192f2f1edd1de7f9d5",
12    "insight_text": "***Spending Pattern:** The transaction indicates a single expense of $20,000 categorized as salary, suggesting it may be an income item misclassified as an expense.\n\n***Money-Saving Tips:**\n\n1. **Review categorization**: Ensure income and expenses are correctly categorized to get an accurate picture of finances.\n\n2. **Create a budget**: Allocate a portion of the corrected income towards savings and essential expenses.\n\n3. **Automate savings**: Set up automatic transfers to a savings or investment account to build wealth over time.",
13    "generated_at": "2026-06-28T20:52:58.723Z",
14    "_v": 0
15   }
16 ]
          
```

Markasread:

The screenshot displays the REST Client interface. The top bar shows the HTTP method 'PUT' and the URL 'http://localhost:5000/api/notifications/6a4173134d547cb70fd6daaf/read'. A 'Send' button is visible. Below the URL bar, tabs for 'Query', 'Headers', 'Auth', 'Body', 'Tests', and 'Pre Run' are shown, with 'Body' selected. The 'Body' tab contains a 'JSON' section with a 'Text' format. The 'JSON Content' area shows a single line of JSON:

```
{
```

. The right sidebar displays the response details: 'Status: 200 OK', 'Size: 28 Bytes', and 'Time: 135 ms'. Below this, tabs for 'Response', 'Headers', 'Cookies', 'Results', and 'Docs' are shown, with 'Response' selected. The 'Response' tab shows a JSON object:

```
{  
  "message": "Marked as read"  
}
```

.

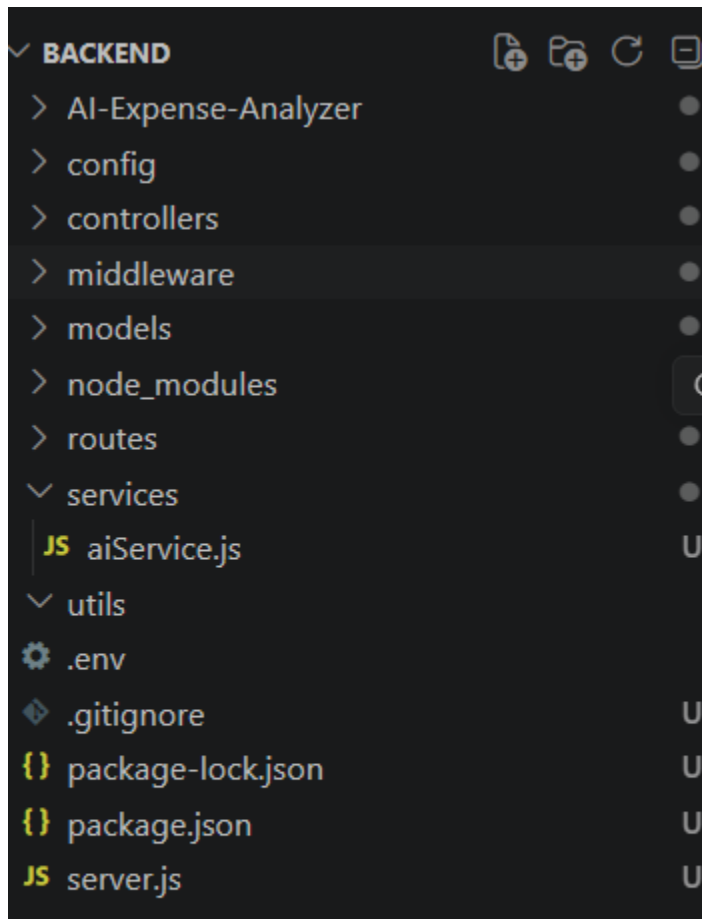
Generate Alignsight:

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:5000/api/ai/generate
- Status:** 201 Created
- Size:** 717 Bytes
- Time:** 907 ms
- Headers:**
 - Content-Type: application/json
 - Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5
- Response (JSON):**

```
1 {
2   "message": "Insight generated",
3   "report": {
4     "userId": "6a418192f2f1edd1de7149d5",
5     "insight_text": "***Spending Pattern:** The transaction indicates a single expense of $20,000
        categorized as salary, suggesting it may be an income item misclassified as an expense.\n\n
        **Money-Saving Tips:**\n\n1. **Review categorization:** Ensure income and expenses are
        correctly categorized to get an accurate picture of finances.\n2. **Create a budget:**
        Allocate a portion of the corrected income towards savings and essential expenses.\n3.
        **Automate savings:** Set up automatic transfers to a savings or investment account to build
        wealth over time.",
6     "_id": "6a4189aa899602cf3e2a0766",
7     "generated_at": "2026-06-28T20:52:58.723Z",
8     "__v": 0
9   }
10 }
```

9. Backend Folder Structure



BACKEND		
> AI-Expense-Analyzer		
< config		
JS db.js		U
< controllers		
JS aiController.js		U
JS aiReportController.js		U
JS authControllers.js		U
JS budgetController.js		U
JS forecastController.js		U
JS notificationController.js		U
JS transactionController.js		U
JS userController.js		U
< middleware		
JS authMiddleware.js		U
< models		
JS AiReport.js		U
JS Budget.js		U
JS Forecast.js		U
JS Notification.js		U
JS Transaction.js		U
JS User.js		U
> node_modules		
< routes		
JS aiRoutes.js		U
JS authRoutes.js		U
JS budgetRoutes.js		U
JS forecastRoutes.js		U
JS notificationRoutes.js		U
JS transactionRoutes.js		U
JS userRoutes.js		U

routes		
JS aiRoutes.js		U
JS authRoutes.js		U
JS budgetRoutes.js		U
JS forecastRoutes.js		U
JS notificationRoutes.js		U
JS transactionRoutes.js		U
JS userRoutes.js		U
< services		
JS aiService.js		U
< utils		
.env		
.gitignore		U
package-lock.json		U
package.json		U
server.js		U

10. GitHub Repository

The first screenshot shows the repository overview for 'AI-Expense-Analyzer' by shakinul-islam. It includes a file list with the following details:

File	Description	Last Commit
backend	Created aiReportController to integrate AI for financial analysis	12 hours ago
database	Create Server.js for running backend	12 hours ago
design	setup for week5	last week
documentation	setup for week5	last week
frontend	update for week 4	2 weeks ago
.gitignore	Updated .gitignore to include backend and environment files	2 weeks ago
README.md	Revise project overview and update directory names	last week

The second screenshot shows the 'database' directory with a commit history table:

Name	Last commit message	Last commit date
..		
aiReport.js	Create aiReport model with schema definition	12 hours ago
Budget.js	Create Budget model with schema for budgeting	12 hours ago
Forecast.js	Create Forecast model with Mongoose schema	12 hours ago
Notification.js	Create Notification model with schema definition	12 hours ago
Server.js	Create Server.js for running backend	12 hours ago
Transaction.js	Create Transaction model for MongoDB using Mongoose	12 hours ago
User.js	Create User model with validation and defaults	12 hours ago
db.js	Add MongoDB connection setup in db.js	12 hours ago

The third screenshot shows the 'backend' directory with a commit history table:

Name	Last commit message	Last commit date
..		
config	Backend: setup backend structure,added core user APIs	12 hours ago
controllers	Created aiReportController to integrate AI for financial analysis	12 hours ago
middleware	Backend: setup backend structure,added core user APIs	12 hours ago
models	Backend: setup backend structure,added core user APIs	12 hours ago
routes	Backend: setup backend structure,added core user APIs	12 hours ago
services	Backend: setup backend structure,added core user APIs	12 hours ago
package-lock.json	Backend: setup backend structure,added core user APIs	12 hours ago
package.json	Backend: setup backend structure,added core user APIs	12 hours ago
server.js	Backend: setup backend structure,added core user APIs	12 hours ago

11. Current Development Progress

Task	Status	Notes
Backend project setup (Node.js + Express)	Completed	Package.json, folder structure initialized
MongoDB connection & configuration	Completed	Mongoose connected, .env configured
Mongoose schema models (all 7 collections)	Completed	All schemas match approved ER Diagram
User registration & login APIs	Completed	bcrypt + JWT implemented
JWT authentication middleware	Completed	Protected routes operational
Transaction CRUD APIs	Completed	Full CRUD with validation
Budget management APIs	Completed	Monthly limit set & retrieve
User profile APIs	Completed	Profile view & update working
Groq API integration service	Completed	Report generation endpoint in testing
AI Reports & Recommendations APIs	Completed	Response caching to be implemented
Notifications & Forecasts APIs	Completed	Basic endpoints created
Global error handling middleware	Completed	Covers all 4xx/5xx responses
API testing via Thunder Client	Completed	Auth & transaction tests passed
Flutter frontend integration	Planned	Week 07 or more objective

12. Reference

- Express.js Documentation — <https://expressjs.com>
- MongoDB Atlas Documentation — <https://www.mongodb.com/docs/atlas/>
- JWT.io — <https://jwt.io>
- bcrypt (npm) — <https://www.npmjs.com/package/bcrypt>
- Thunder Client — <https://www.thunderclient.com>
- Groq API Documentation — <https://console.groq.com/docs>

GitHub Repository

<https://github.com/shakinul-islam/AI-Expense-Analyzer>